



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

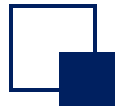
規格嚴格 功夫到家
1920 — 2017

Introduction to Object-Oriented Software Construction



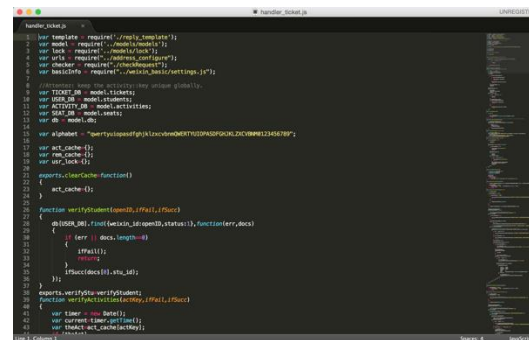
Chapter I

- Software construction basic process and goals
- Object-oriented thinking
- Design patterns
- Introduction to the Java language
- Java development environment construction and program examples



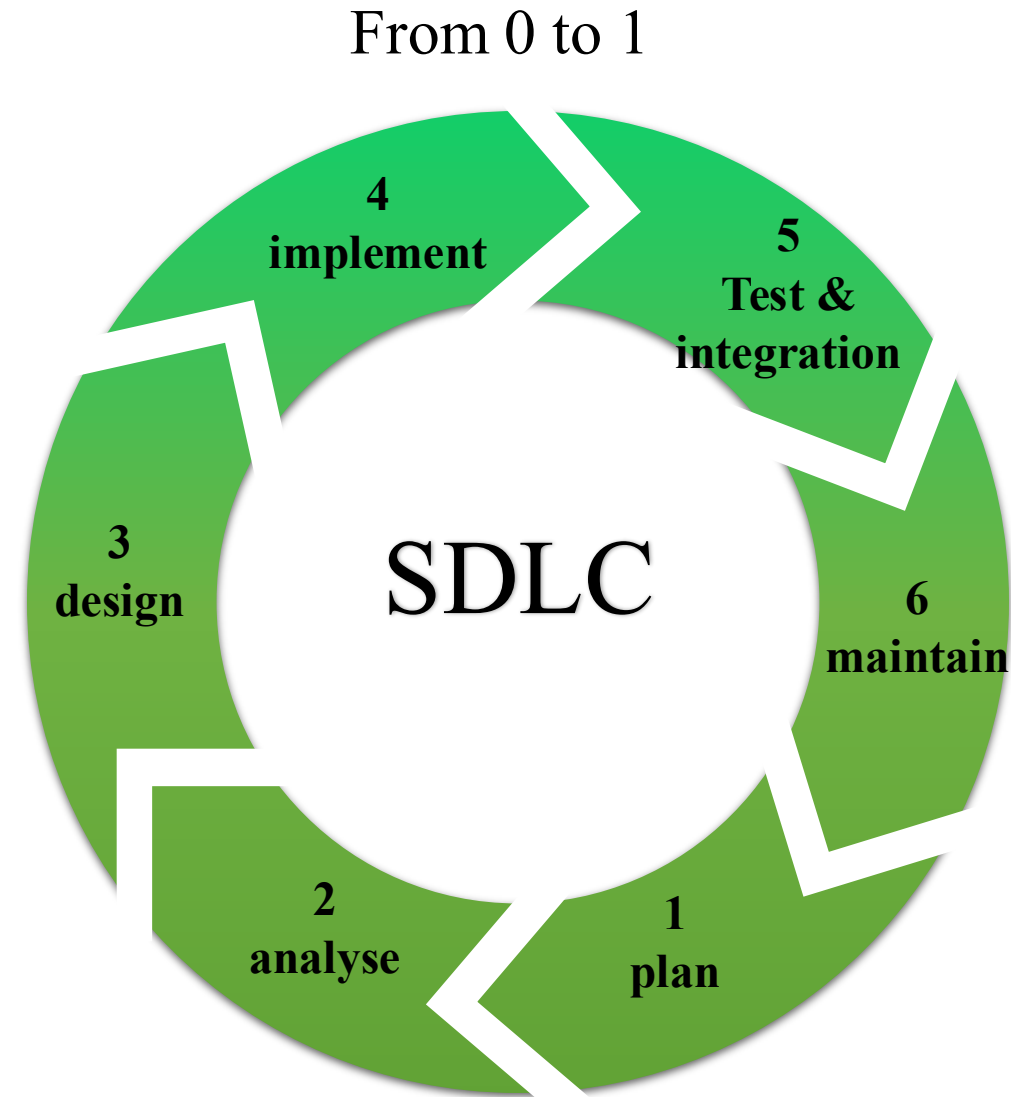
What is software?

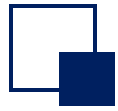
- Software = program + data + documentation
 - Program: A set of instructions that the computer can accept and the runtime can provide the required functionality and performance.
 - Data: Data structures that enable programs to manipulate information appropriately.
 - Documentation: Describing the development process, methods and graphic materials used in the program.



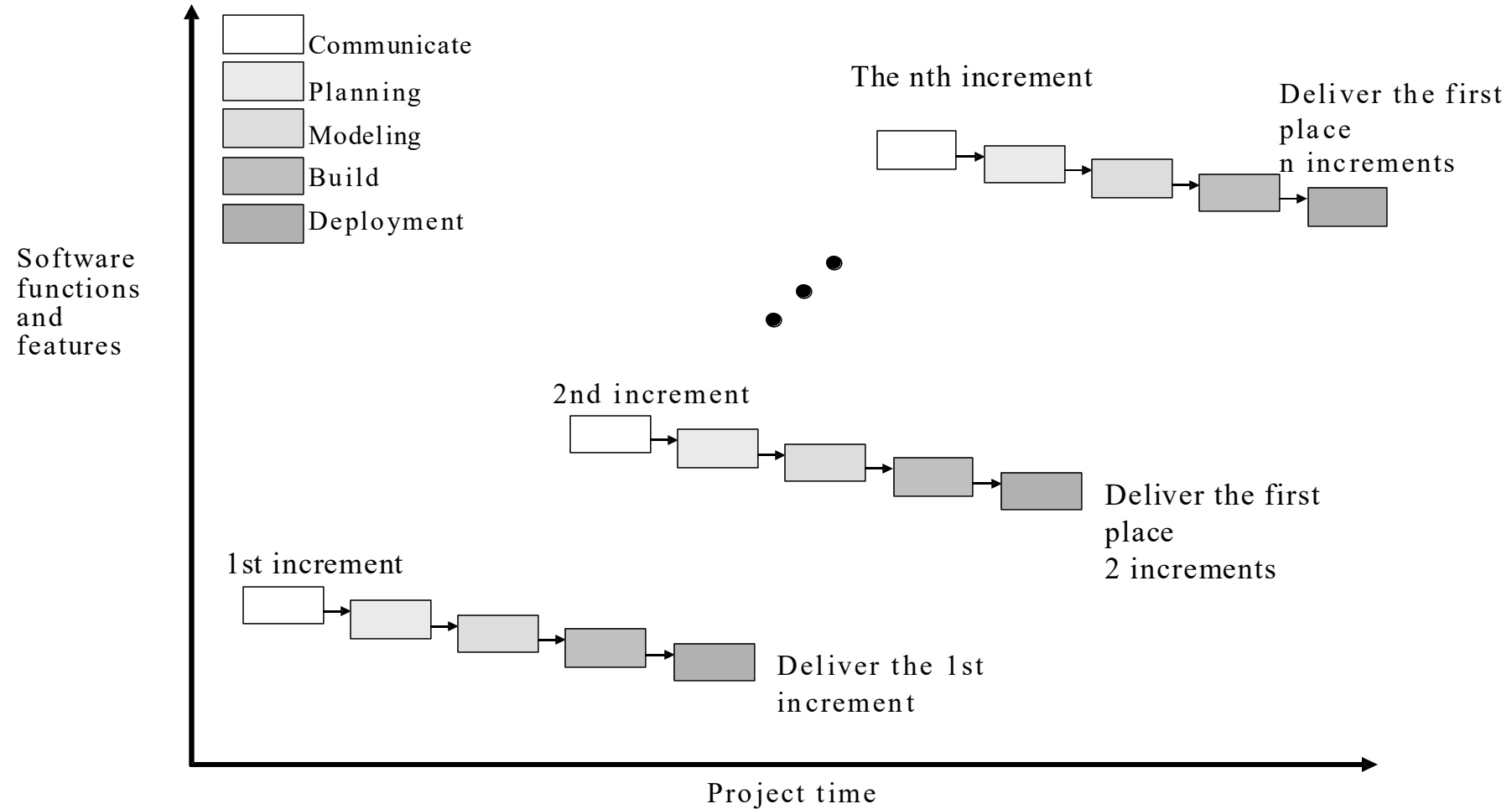


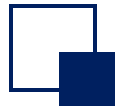
Software Development Life Cycle (SDLC)





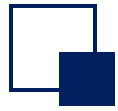
Incremental process





Prototyping process

- The prototyping process is a program that can be quickly established and can run on a computer, and the functions it can accomplish are often a subset of what the final product can accomplish. The role of prototypes in development:
 - (1) Obtain the real needs of users
 - (2) It can be used to determine the technical, cost, and schedule possibilities for a project or parts of a project
- Iterative techniques are employed as prototype systems are continuously adapted to meet the needs of various stakeholders, while also allowing developers to gradually understand the needs of users.



Test-driven development

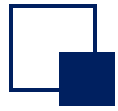
Test-Driven Development (TDD) is a new development method that is different from the traditional software development process. It requires that **the test code be written before writing the code for a function**, and then only the functional code that passes the test is written, and the test is used to promote the entire development. This helps to write concise, usable and high-quality code and accelerates the development process.



Test-driven development

□ advantage

- Strengthening the understanding of requirements helps to design easy-to-use interfaces from the user's perspective.
- Strengthening the concept of modules and interfaces has led us to implement loosely coupled designs and rely more on interfaces to improve the scalability and resilience of the system.
- Run unit tests frequently to detect errors early and greatly reduce the cost of subsequent testing and fixing.
- Continuous regression testing is provided, and the code is easy to maintain and refactor.
- Test code that verifies functional correctness in real time is the best code documentation and is synchronized with the latest code.
- Reduce stress, reduce worry, increase confidence in our code, and improve program development efficiency.



The goal of the software construction

- **Comprehensibility**

- Follow specific naming conventions
- There are sufficient notes and explanations
- The code layout is reasonable:
indent/blank/chunking/aligning, *etc*
- The document is organized in a reasonable way
- The method should not be too long or too short, and
should be easy to understand
- Avoid multi-layer nesting - increase complexity

- **Maintainability**

- When software changes, can you adapt to the change at
a small cost?
- Modular programming, high cohesion and low
coupling
- Use design patterns flexibly

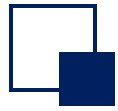
Example #a:

$$z = ((3*x^2) + (4*x) - 5) - ((2*y^2) - (7*y) + 11) / ((3*x^2) + (4*x) - 5)$$

Example #b:

$$\begin{aligned} a &= ((3*x^2) + (4*x) - 5) \\ b &= ((2*y^2) - (7*y) + 11) \\ z &= (a - b) / a \end{aligned}$$





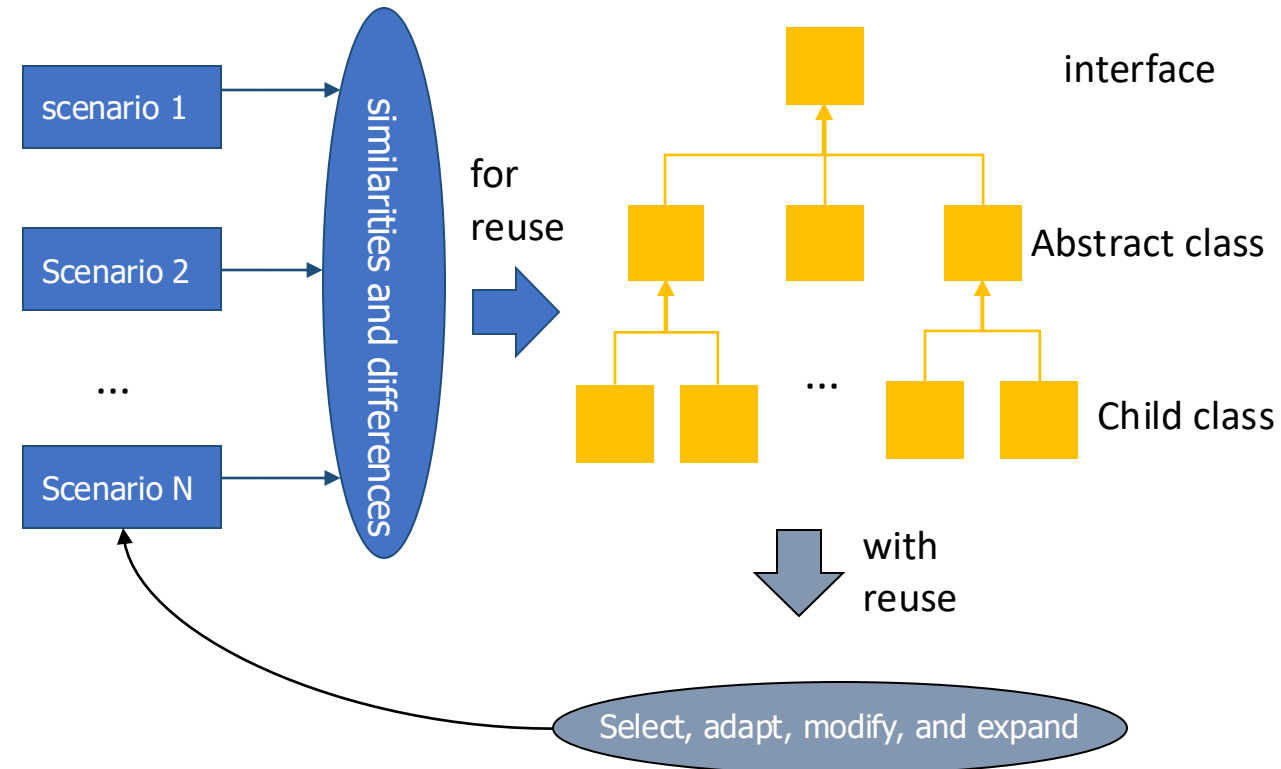
The goal of the software construction

- **Reusability**

- Construct software modules/APIs that can be reused in different applications
- Modular, generic, abstract
- Provide complete fine-grained operations to ensure functional integrity and reuse different operations in different scenarios

- **Spatiotemporal performance**

- Memory performance
- Algorithm performance
- I/O performance



Which of the following is not an advantage of test-driven development?

- ☐ A It is conducive to designing interfaces that are easy to use
- ☐ B It is suitable for rapid iterative development and flexibly handles changes in requirements
- ☐ C Easy to refactor, good for code maintenance
- ☐ D Frequent testing can detect errors early

Which of the following is not an advantage of test-driven development?

- ☐ A It is conducive to designing interfaces that are easy to use
- ☒ B It is suitable for rapid iterative development and flexibly handles changes in requirements
- ☐ C Easy to refactor, good for code maintenance
- ☐ D Frequent testing can detect errors early



Software engineering methodology

- **Structured Approach (Process-Oriented)**

- Complex world → complex process (the occurrence and development of things)
- Design a set of features (or algorithms) to solve a problem
- Find the right way to store and process data

- **Object-oriented approach**

- Any system is made up of objects that can accomplish a set of related tasks
- If an object depends on a task that is not for which it is responsible, then it needs to access another object that is responsible for this task (calling the method of another object)
- An object cannot directly manipulate the data inside another object, nor can it give other objects direct access to its own data
- All communication must be called by method



Object-oriented three characteristics

❑ Encapsulation:

- hide the properties and implementation details of the object, and only publicly access the method;
- Enhanced security and simplified programming

student

- Private information and operations
- Open action

❑ Inheritance

- Subclasses inherit the characteristics and behaviors of their parents
- Implement code reuse

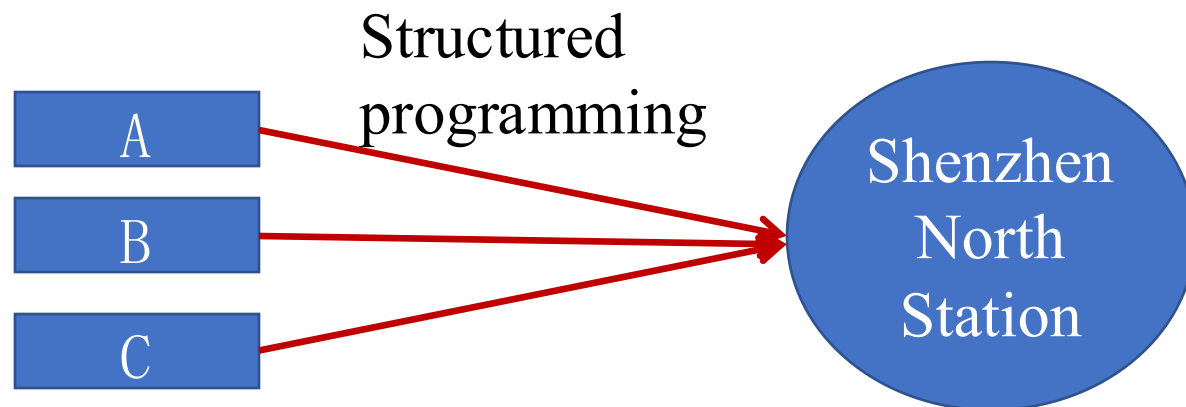
- Students - CS students

❑ Polymorphism

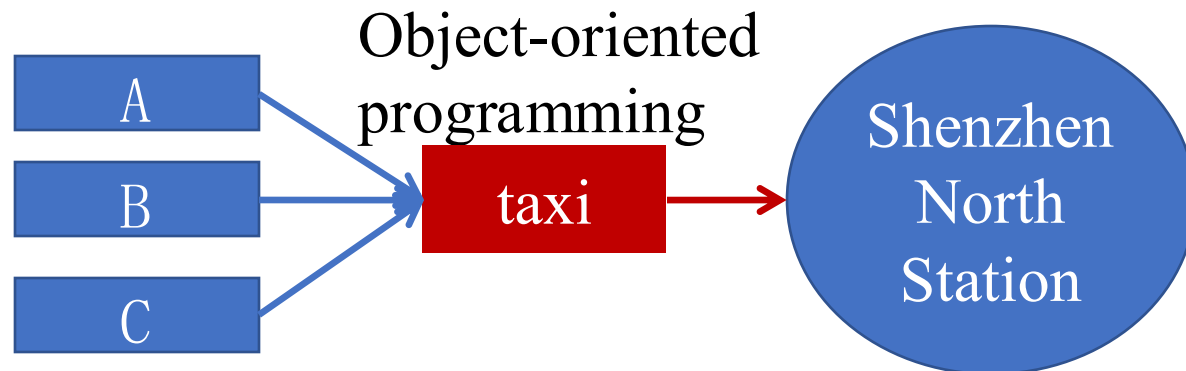
- The ability of the same behavior to have multiple different manifestations ("one interface, multiple methods")
- Improved program scalability and maintainability

- Course:
 - School of Computer Science - Compilation Principles
 - School of Science - Physics

Advantages of OOP: decoupling, encapsulation



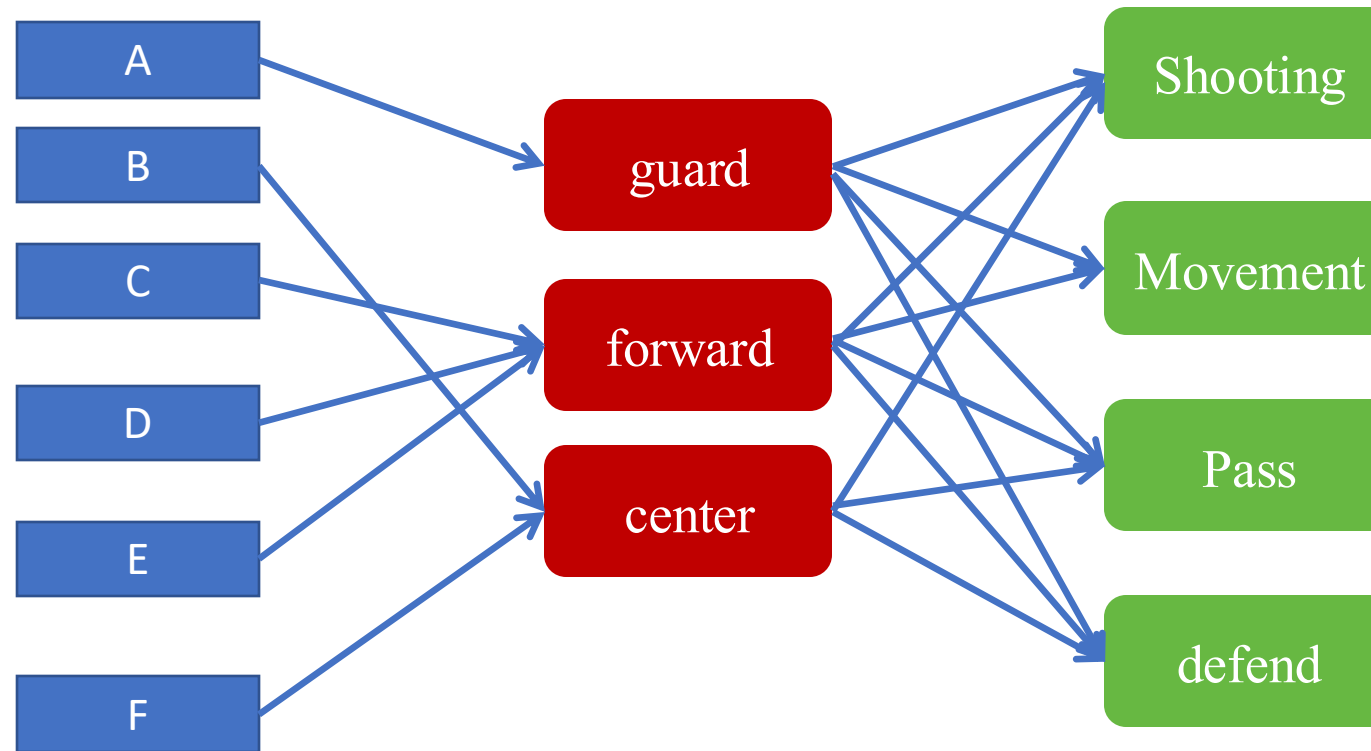
In structured programming, everyone should include the method of getting to Shenzhen North Railway Station if there is a change, such as "Liuxian Avenue road construction". You need to change everyone's program code.



In object-oriented programming, the process of going to the North Station is "encapsulated" separately by creating an object "taxi". It realizes the "decoupling" of ABC's method to Shenzhen North Railway Station. If you want to change it, just change the method of the taxi class.

Advantages of OOP: reuse

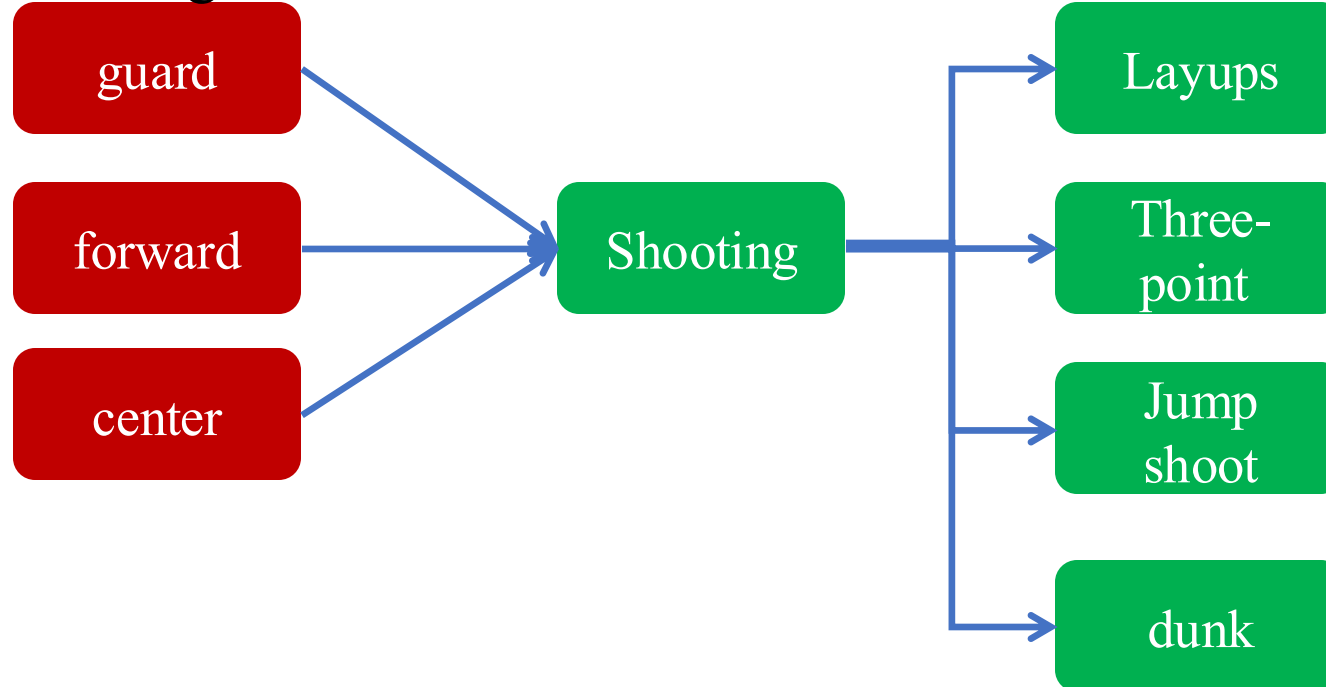
- Structured programming: Each new player is constructed, and his series of actions such as shooting, running, passing, and defending are coded.
- Object-oriented programming: Reuse the previous code modules according to the "class" to which the new team member belongs. You can even quickly create new player types.

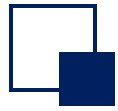


Advantages of OOP: Polymorphism

Polymorphism in object-oriented programming allows for different implementations of the same interface, with the advantage of separating the implementation details from the upper objects.

Imagine that a coach on the basketball court only needs to shout "shoot" to the team members, and they can execute this "shooting" instruction in a pre-designed way and with a variety of different strategies.





The concept of design patterns

- Design Pattern

- It is a summary of the design experience of code that has been used repeatedly and has been classified and cataloged. It describes some of the recurring problems in the software design process, as well as the core solution to that problem.
- The purpose is to improve code reusability, code readability, and code reliability。

□ The essence of the design pattern is the practical application of object-oriented design principles, and the full understanding of the encapsulation, inheritance and polymorphism of classes, as well as the relationship and combination relationship of classes.

□ note

- An abstract scheme
- Schemas are not specific libraries, macro definitions, template classes



The main differences between Java and C/C++

- Cross-platform
 - C/C++ language is compiled into machine code,
 - Java language is compiled into bytecode, and machine code is compiled through the JVM.
- Abolition of pointers
 - C/C++ languages have pointer types,
 - Java languages do not.
- inherit
 - C++ allows multiple inheritance,
 - Java language only allows single inheritance.
- speed
 - C/C++ runs fast,
 - Java runs slowly.



The basic process of running a Java program

- Write Java source programs using the editor
 - Source file name: The main class name .java
- Use the compiler (javac) to compile the source program into bytecode
 - Bytecode file name: source file name .class
- Run with a virtual machine (interpreter, java).
 - Running process: loading, code verification, interpretation and execution
 - For example, the java source filename .class

The three main characteristics of OOP are:

- A encapsulation
- B inheritance
- C modularization
- D Polymorphism
- E Streamline

The three main characteristics of OOP are:

- ☒ A encapsulation
- ☒ B inheritance
- ☐ C modularization
- ☒ D Polymorphism
- ☐ E Streamline

submit